

Enterprise Disaster Recovery & Business Continuity Plan

Universal Device & Service Monitoring Platform — Zero-Cost 3-2-1 Architecture

Target Domain: Hybrid Cloud & Edge Network	Document Version: v1.0.0 (Enterprise Production)
Repository: github.com/erdipu/enterprise-infrastructure-monitoring	Production Host: deepak-monitoring.duckdns.org (80.225.252.145)
RTO (Recovery Time): < 15 min (PC) < 30 min (Cloud)	RPO (Recovery Point): < 24h (Nightly DB) Instant (Git)
Backup Architecture: 3-2-1 Zero-Cost Storage	Security Policy: Sanitized — Zero Secrets Committed

1. Executive Overview & Business Continuity Objectives

This specification establishes the authoritative Disaster Recovery Plan (DRP) and business continuity framework for the Universal Device & Service Monitoring Platform. The objective is to guarantee **100% operational restoration and seamless development resumption** even in catastrophic events, including local PC loss, disk formatting, cloud instance termination, database corruption, or complete loss of AI conversation transcripts.

Guaranteed Recovery Commitments:

- Independent Rebuild:** Any engineer can deploy the platform from scratch on a brand-new machine in under 15 minutes.
- Zero Secrets Leakage:** Sensitive API tokens, private SSH keys, and database passwords are never exposed publicly on Git.
- Zero-Cost 3-2-1 Strategy:** Full disaster resilience is achieved using Oracle Cloud Always Free, GitHub, and free cloud storage.
- Verified Data Persistence:** Automated nightly database backups with 30-day retention and automated cryptographic verification.

2. Complete GitHub Backup & Resource Inventory

All source code, telemetry daemons, database DDLs, monitoring rules, and engineering runbooks are centrally version-controlled in the repository: <https://github.com/erdipu/enterprise-infrastructure-monitoring.git>.

Component	Files / Path	Role & Content	Status
Backend API	backend/	Flask REST API, 8 SQLAlchemy ORM models, auth routes, health endpoints	100% Backed Up
Frontend UI	frontend/	HTML5/CSS3/Vanilla JS real-time operations dashboard, incident tables	100% Backed Up
Edge Daemons	router_syslog_exporter.py router_watchdog.py	UDP 514 syslog ingest, :9125 telemetry exporter, 7-stage watchdog engine	100% Backed Up
Database DDL	database/schema.sql database/init.sql	Complete 8-table PostgreSQL DDL, foreign keys, indexes, and seed records	100% Backed Up
Config Templates	.env.example alert_config.example.json	Sanitized parameter templates without real passwords, bot tokens, or keys	Sanitized
Monitoring Stack	prometheus/ alertmanager/	Scrape configs, Blackbox probes, relabeling, and alert routing rules	100% Backed Up
DR Scripts	scripts/*.sh scripts/*.py	PostgreSQL backup/restore engines, GZIP/SHA-256 verifiers, and offline packager	Tested & Verified
Documentation	docs/*.md README.md	6 DR guides, 6 ITIL SOP runbooks, setup manual, and system architecture PDF	Complete

3. 3-2-1 Zero-Cost Backup Architecture

To ensure complete resilience against concurrent local and cloud failures without ongoing subscription costs, the platform strictly adheres to the **3-2-1 Backup Rule**:

Storage Tier	Storage Medium	Backup Content & Mechanism	Cost
Copy 1 (Primary Live)	OCI Cloud VM 2 (80.225.252.145)	Live production containers, PostgreSQL volume, automated nightly cron snapshots in /home/ubuntu/.../backups	INR 0 (Always Free)
Copy 2 (Version Control)	GitHub Remote (Git Cloud)	Source code, DDL schemas, configurations, runbooks, documentation, and architecture PDF on branch main	INR 0 (Free Tier)
Copy 3 (Cold Offsite)	Encrypted Cloud Drive & Air-Gapped USB	Offline Git bundles (.bundle), source tarballs, AES-256 encrypted credential vaults, and verified SQL snapshots	INR 0 (Google Drive 15GB + USB)

Offline Standalone Packaging Utility

Running bash scripts/create_offline_bundle.sh creates a standalone offline package containing a complete Git bundle (100% history cloneable without internet) and clean source tarball with SHA-256 checksums.

```
# Generate Standalone Offline Bundles (Runs on macOS, Linux, or Cloud VM):
bash scripts/create_offline_bundle.sh
# Output generated in dist_backups/:
• enterprise_monitoring_git_<TIMESTAMP>.bundle (Full Git history)
• enterprise_monitoring_full_<TIMESTAMP>.tar.gz (Clean source tree)
• *.sha256 (Cryptographic verification signatures)
```

4. Step-by-Step PC Rebuild Guide (< 15 Minutes RTO)

If your computer is formatted, damaged, or replaced with a completely new workstation, follow this exact sequence to restore access and resume engineering operations:

Phase	Action & Commands	Execution Target
1. Base Tools	macOS: brew install git python3 docker Windows: Install Git for Windows, Docker Desktop, Python 3.11+ Ubuntu: sudo apt update && sudo apt install -y git python3 docker.io docker-compose-v2	New Local PC
2. Clone Repo	git clone https://github.com/erdipu/enterprise-infrastructure-monitoring.git cd enterprise-infrastructure-monitoring	Terminal / Shell
3. Config Setup	cp .env.example .env && cp alert_config.example.json alert_config.json Populate API tokens and passwords from Bitwarden Vault or encrypted backup.	Repo Root
4. Cloud SSH Key	Copy backed-up ssh-key-2026-10-08.key into ~/.ssh/ and run: chmod 600 ~/.ssh/ssh-key-2026-10-08.key	~/.ssh/ directory
5. Verify Cloud	ssh -i ~/.ssh/ssh-key-2026-10-08.key ubuntu@80.225.252.145 "docker ps" Confirms production server connectivity and verifies 7 containers are healthy.	Remote OCI VM
6. Local Stack	docker compose up -d Spins up local PostgreSQL, Prometheus, Grafana, Alertmanager, and Backend API.	Local Docker

5. Disaster Recovery Scenarios & Action Matrix

Standard operating remediation procedures for catastrophic infrastructure and hardware failure scenarios:

Failure Scenario	Business Impact	Standard Recovery Action Plan	RTO
A. PC Destroyed / Lost	Zero local code or tools	Acquire new PC, follow Section 4 Rebuild Guide, clone GitHub repo, restore SSH key from Bitwarden.	< 15 min
B. Cloud VM Terminated	Production server down	Launch new Ubuntu VM on Oracle/AWS, follow docs/PROJECT_SETUP_GUIDE.md, restore database snapshot using scripts/restore_database.sh.	< 30 min
C. Database Corrupted	Table data or schemas lost	Execute ./scripts/restore_database.sh backups/eim_db_backup_latest.sql.gz. Takes automatic pre-restore safety dump, drops locks, and restores 8 tables.	< 2 min
D. GitHub Unavailable	Remote repo inaccessible	Clone from offline Git bundle: git clone dist_backups/enterprise_monitoring_git_*.bundle repo. All branches and commits are restored immediately.	< 3 min
E. Secrets / Key Lost	Loss of cloud access	Access Bitwarden Vault or decrypt platform_secrets_backup.enc. Regenerate SSH key via Oracle Cloud Console Serial Console if needed.	< 10 min
F. AI History Deleted	Lost chat transcript	100% of architecture, commands, and operations are preserved inside README.md, docs/, and this specification document.	0 min

6. Database Backup, Retention & Resilient Restoration

The production database monitoring_db runs on PostgreSQL 15 and hosts 8 mission-critical tables: users, servers, monitoring_targets, incidents, alerts, incident_comments, audit_logs, and notifications.

Automated Nightly Snapshotting & Retention

An automated cron daemon executes nightly at 02:00 AM UTC on Cloud VM 2: 0 2 * * * /home/ubuntu/.../scripts/backup_database.sh >> /home/ubuntu/.../backups/backup.log 2>&1. Backups are compressed with gzip and automatically purged after 30 days.

```
# 1. Verify Backup Integrity (GZIP Decompression, 4/4 SQL Keywords, SHA-256):
python3 scripts/verify_backup_integrity.py backups/eim_db_backup_20261010_121711.sql.gz
[+] File size: 8.60 KB | GZIP: Valid | Keywords: 4/4 Found | SHA-256: 5db8d5... | PASSED

# 2. Resilient Database Restoration (Includes Auto-Safety Backup & Lock Release):
./scripts/restore_database.sh backups/eim_db_backup_20261010_121711.sql.gz
[*] Auto-creates safety snapshot: backups/safety_snapshots/pre_restore_safety_<ts>.sql.gz
[*] Terminates active client connections gracefully via pg_terminate_backend
[*] Rebuilds schema & imports data | Table count verified: 8 tables active
```

7. Credentials Vault & Administrative Access Recovery

To protect infrastructure against credential theft while ensuring zero permanent lockouts:

Credential Item	Purpose & Service	Secure Storage Location	Disaster Recovery Protocol
OCI SSH Private Key	ssh-key-2026-10-08.key Access to Oracle VM 2	Bitwarden Secure Note & Encrypted USB flash drive	Restore key text to ~/.ssh/, set chmod 600. Or access via OCI Cloud Web Serial Console.
Telegram Bot Token	@my_infrastructure_alert_bot Real-time incident alerts	Bitwarden Secure Note & alert_config.json	Query Telegram @BotFather with /token to view or revoke/regenerate immediately.
GitHub Access Token	Personal Access Token (PAT) Git push authentication	Bitwarden Password Manager	Sign in to GitHub via 2FA authenticator app, go to Settings > Developer Settings > Tokens to generate new PAT.
DuckDNS Token & SMS	Dynamic DNS updates & Fast2SMS API engine	Bitwarden Secure Note	Log in to duckdns.org or fast2sms.com via email OTP to retrieve API keys instantly.

Encrypted Offline Secret Archive Command

To back up sensitive files into an AES-256 encrypted archive for USB cold storage: `tar -czf - .env alert_config.json | openssl enc -aes-256-cbc -pbkdf2 -salt -out platform_secrets.enc`. Decrypt anytime using: `openssl enc -d -aes-256-cbc -pbkdf2 -in platform_secrets.enc | tar -xzf -`.

8. Emergency Disaster Checklist & Verification Drills

Print or maintain this action checklist for emergency recovery operations:

[] 1. CLONE REPOSITORY: `git clone https://github.com/erdipu/enterprise-infrastructure-monitoring.git`

[] 2. RESTORE ENVIRONMENT: Copy `.env.example` to `.env` and `alert_config.example.json` to `alert_config.json`

[] 3. RETRIEVE SECRETS: Retrieve API tokens and bot credentials from Bitwarden or decrypt `platform_secrets.enc`

[] 4. RESTORE SSH KEY: Place `ssh-key-2026-10-08.key` in `~/.ssh/` and run `chmod 600 ~/.ssh/ssh-key-2026-10-08.key`

[] 5. VERIFY PRODUCTION VM: `ssh -i ~/.ssh/ssh-key-2026-10-08.key ubuntu@80.225.252.145 "docker ps && crontab -l"`

[] 6. VERIFY TELEGRAM ALERTS: Check alert endpoints at `http://deepak-monitoring.duckdns.org:9093/#!/alerts`

[] 7. VERIFY LOCAL ENGINE: Run `docker compose up -d` to verify local development stack

Prepared By: Lead SRE / Infrastructure Operations	Classification: Confidential — Engineering Operations
Status: Production Active & Live Tested	Audit Date: October 10, 2026